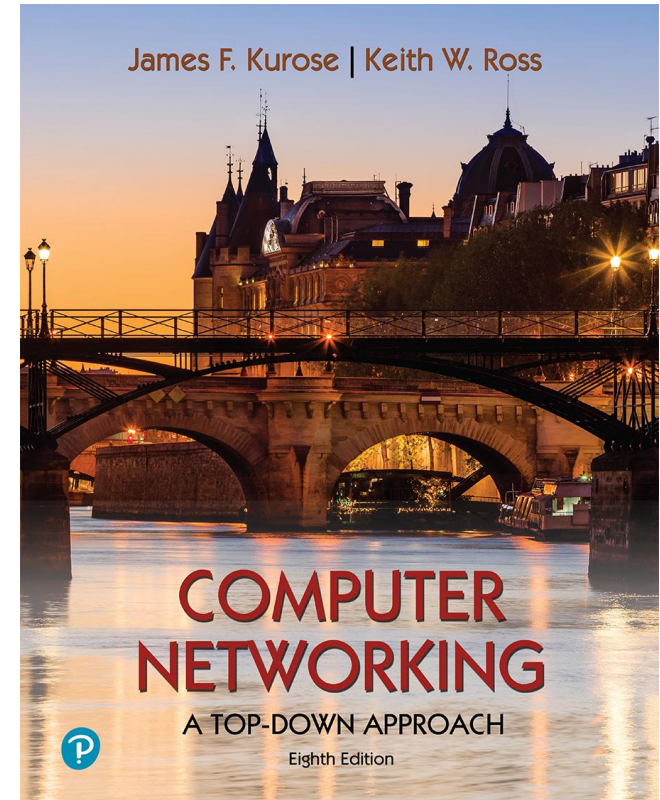


Chương 9

Tầng Ứng dụng



Computer Networking: A Top-Down Approach

8th edition

Jim Kurose, Keith Ross
Pearson, 2020

Tầng Ứng dụng: Tổng quan

- Nguyên lý của ứng dụng mạng
- Web và HTTP
- E-mail, SMTP, IMAP
- Hệ thống tên miền DNS



Tầng Ứng dụng: Tổng quan

Mục tiêu:

- Các khía cạnh về khái niệm và thực thi của các giao thức tầng ứng dụng.
 - các mô hình dịch vụ tầng vận chuyển
 - mô hình client-server
 - mô hình peer-to-peer
- Tìm hiểu về các giao thức bằng cách xem xét các giao thức tầng ứng dụng và cơ sở hạ tầng phổ biến.
 - HTTP
 - SMTP, IMAP
 - DNS

Một số ứng dụng mạng

- mạng xã hội
 - Web
 - tin nhắn văn bản
 - e-mail
 - trò chơi mạng nhiều người dùng
 - phát video được lưu trữ dưới dạng luồng (streaming).
(YouTube, Hulu, Netflix)
 - chia sẻ tệp tin ngang hàng (P2P)
 - âm thanh qua IP (ví dụ: Skype)
 - hội nghị truyền hình thời gian thực (ví dụ: Zoom)
 - tìm kiếm Internet
 - đăng nhập từ xa
 - ...
- Q: yêu thích của bạn?*

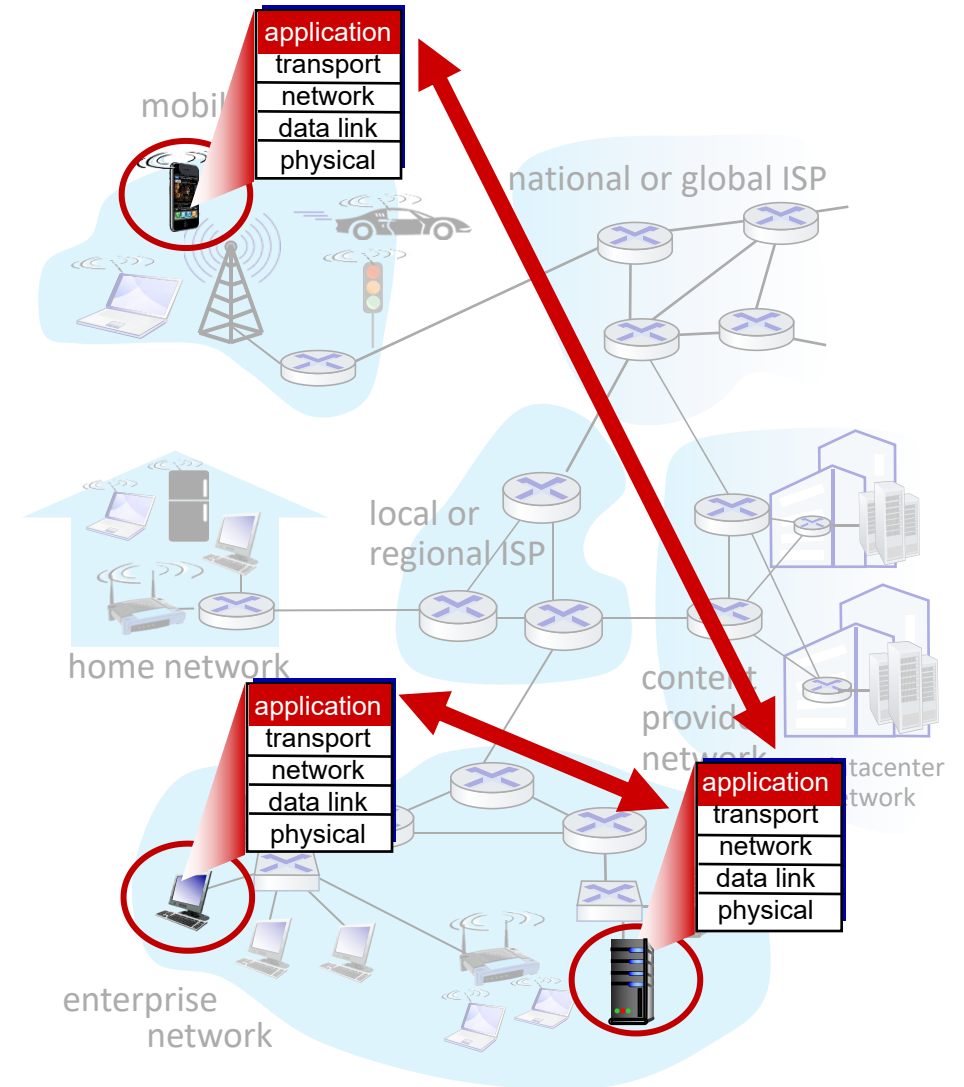
Tạo một ứng dụng mạng

viết chương trình để:

- chạy trên các hệ thống đầu cuối (khác nhau)
- giao tiếp qua mạng
- ví dụ: phần mềm máy chủ web giao tiếp với phần mềm trình duyệt

không cần viết phần mềm cho các thiết bị lõi mạng/network-core

- thiết bị lõi mạng không chạy ứng dụng người dùng
- các ứng dụng trên hệ thống đầu cuối cho phép phát triển, phổ biến ứng dụng nhanh chóng



Mô hình client-server

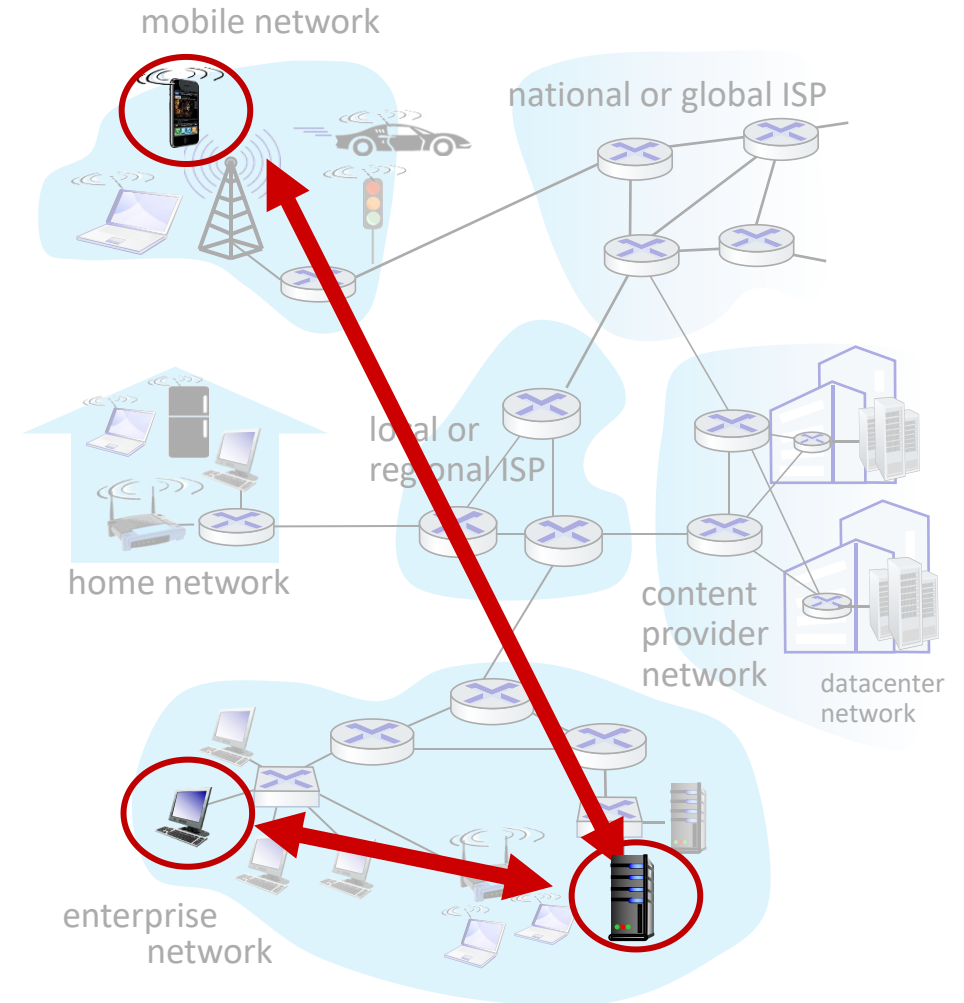
server/máy chủ:

- máy chủ luôn được bật
- địa chỉ IP cố định
- thường đặt tại các trung tâm dữ liệu để tăng khả năng mở rộng

clients/máy khách:

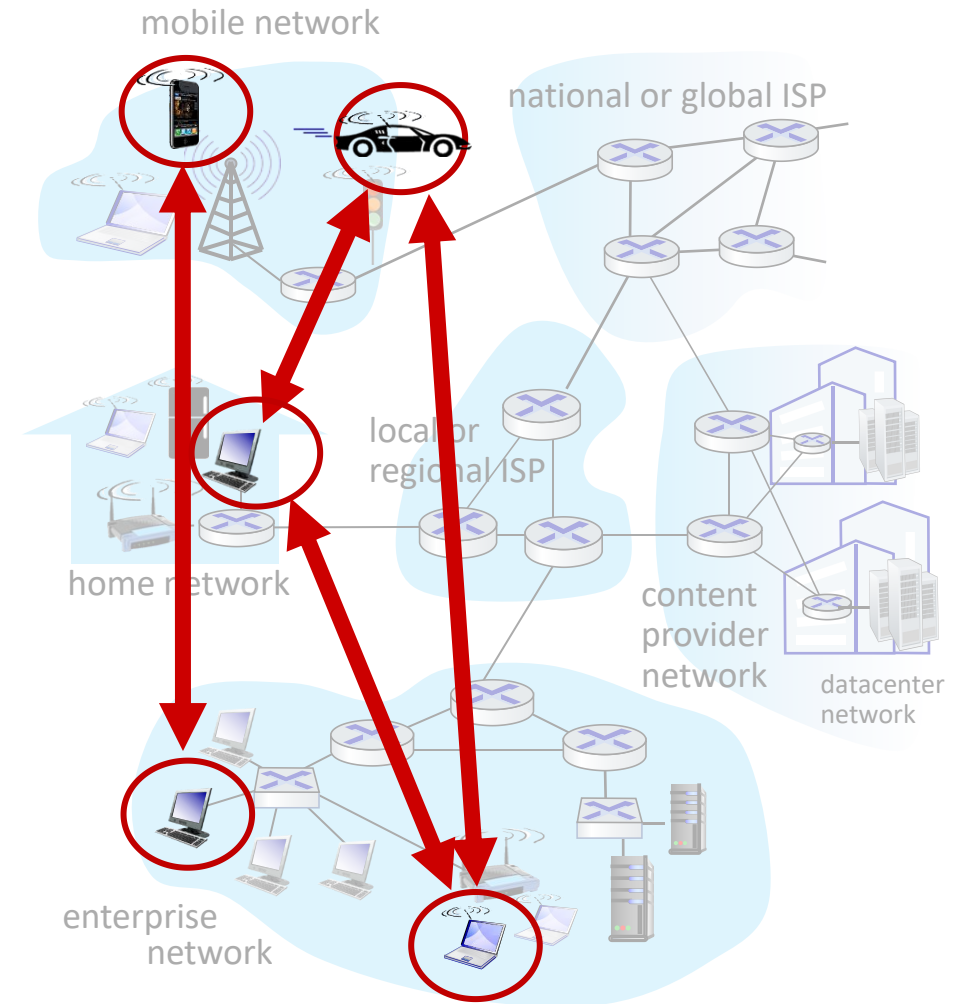
- liên hệ, giao tiếp với máy chủ
- có thể được kết nối không liên tục
- có thể có địa chỉ IP động
- không giao tiếp trực tiếp với nhau

ví dụ: HTTP, IMAP, FTP



Kiến trúc ngang hàng (Peer-peer architecture)

- không có máy chủ luôn bật
- hệ thống đầu cuối tùy ý giao tiếp trực tiếp
- peers yêu cầu dịch vụ từ các peers khác, đồng thời cung cấp dịch vụ cho peers khác
 - *tự mở rộng* - peers mới mang lại khả năng dịch vụ mới, cũng như nhu cầu dịch vụ mới
- các người dùng kết nối không liên tục và thay đổi địa chỉ IP
 - quản lý phức tạp
- ví dụ: Chia sẻ tệp P2P



Giao tiếp giữa các tiến trình

Process/tiến trình: chương trình chạy bên trong máy chủ/host

- trong cùng một host, hai tiến trình giao tiếp bằng cách sử dụng **giao tiếp giữa các tiến trình** (được định nghĩa bởi HĐH)
- các tiến trình trong các host khác nhau giao tiếp bằng cách trao đổi **thông điệp/messages**

clients, servers

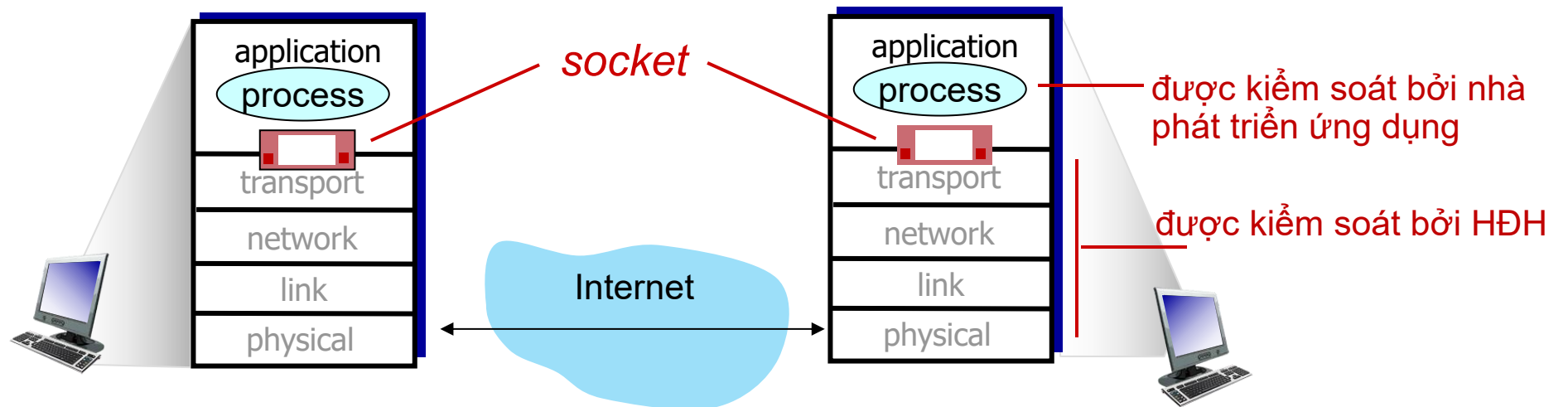
client process: tiến trình khởi tạo giao tiếp

server process: tiến trình đợi được liên hệ

- Ghi chú: các ứng dụng có kiến trúc P2P có các tiến trình client và các tiến trình server

Sockets/Ổ cắm

- tiến trình gửi/nhận thông điệp đến/từ **socket** của nó
- Ổ cắm tương tự như cửa
 - tiến trình gửi đẩy tin nhắn ra khỏi cửa
 - tiến trình gửi thông điệp phụ thuộc vào cơ sở hạ tầng vận chuyển ở phía bên kia cửa để gửi thông điệp đến socket của tiến trình nhận
 - Hai socket có liên quan: một socket ở mỗi bên



Địa chỉ hóa các tiến trình

- để nhận thông điệp, tiến trình phải có *định danh*
- thiết bị chủ có địa chỉ IP 32 bit duy nhất
- Q: địa chỉ IP của host mà tiến trình chạy trên đó có đủ để xác định tiến trình không?
 - A: không, nhiều tiến trình có thể chạy trên cùng một host
- *định danh* bao gồm cả địa chỉ IP và số cổng được liên kết với tiến trình trên host.
- Ví dụ về số cổng/port numbers:
 - HTTP server: 80
 - mail server: 25
- Để gửi thông điệp HTTP đến gaia.cs.umass.edu web server:
 - *IP address*: 128.119.245.12
 - *port number*: 80

Một giao thức tầng ứng dụng định nghĩa:

- các loại thông điệp được trao đổi,
 - ví dụ: yêu cầu, phản hồi
- cú pháp thông điệp:
 - các trường trong thông điệp và cách phân cách các trường.
- ngữ nghĩa thông điệp
 - ý nghĩa của thông tin trong các trường dữ liệu
- quy tắc cho việc khi nào và cách các tiến trình gửi và phản hồi thông điệp thế nào

các giao thức mở:

- được định nghĩa trong các tài liệu RFC, tất cả mọi người đều có quyền truy cập vào định nghĩa giao thức
- cho phép khả năng tương kết/tương thích
- ví dụ: HTTP, SMTP

các giao thức độc quyền:

- ví dụ: Skype, Zoom

Ứng dụng cần dịch vụ vận chuyển nào?

tính toàn vẹn dữ liệu:

- một số ứng dụng (ví dụ như truyền tập tin, giao dịch web) yêu cầu truyền dữ liệu tin cậy 100%
- Các ứng dụng khác (ví dụ như âm thanh) có thể chấp nhận mất mát một số dữ liệu.

timing/định thời

- một số ứng dụng (ví dụ như điện thoại Internet, trò chơi tương tác) yêu cầu độ trễ thấp để có “hiệu quả”

throughput/thông lượng

- một số ứng dụng (ví dụ: đa phương tiện) yêu cầu lượng thông lượng tối thiểu nào đó để “hiệu quả”
- các ứng dụng khác (“ứng dụng mềm dẻo”) tận dụng bất kỳ thông lượng nào mà chúng nhận được

security/bảo mật

- mã hóa, tính toàn vẹn dữ liệu,...

Yêu cầu dịch vụ vận chuyển: các ứng dụng phổ biến

ứng dụng	mất dữ liệu	thông lượng	nhạy cảm thời gian?
file transfer/download	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5Kbps-1Mbps video:10Kbps-5Mbps	yes, 10's msec
streaming audio/video	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	Kbps+	yes, 10's msec
text messaging	no loss	elastic	yes and no

Các dịch vụ giao thức vận chuyển Internet

dịch vụ TCP:

- *vận chuyển* tin cậy giữa tiến trình gửi và nhận
- *kiểm soát luồng*: bên gửi sẽ không làm ngập bên nhận
- *kiểm soát tắc nghẽn*: điều tiết bên gửi khi mạng quá tải
- *hướng kết nối*: yêu cầu thiết lập kết nối giữa các tiến trình client và server
- *không cung cấp*: định thời, đảm bảo thông lượng tối thiểu, bảo mật

dịch vụ UDP:

- *truyền dữ liệu không tin cậy* giữa tiến trình gửi và nhận
- *không cung cấp*: độ tin cậy, kiểm soát luồng, kiểm soát tắc nghẽn, định thời, đảm bảo thông lượng, bảo mật hoặc thiết lập kết nối.

Q: tại sao bận tâm? Tại sao lại có UDP?

Các ứng dụng Internet và các giao thức vận chuyển

ứng dụng	giao thức tầng ứng dụng	giao thức vận chuyển
file transfer/download	FTP [RFC 959]	TCP
e-mail	SMTP [RFC 5321]	TCP
Web documents	HTTP 1.1 [RFC 7320]	TCP
Internet telephony	SIP [RFC 3261], RTP [RFC 3550], or proprietary	TCP or UDP
streaming audio/video	HTTP [RFC 7320], DASH	TCP
interactive games	WOW, FPS (proprietary)	UDP or TCP

Application layer: overview

- Principles of network applications
- **Web and HTTP**
- E-mail, SMTP, IMAP
- The Domain Name System
DNS



Web and HTTP

First, a quick review...

- web page consists of *objects*, each of which can be stored on different Web servers
- object can be HTML file, JPEG image, Java applet, audio file,...
- web page consists of *base HTML-file* which includes *several referenced objects, each* addressable by a *URL*, e.g.,

`www.someschool.edu/someDept/pic.gif`

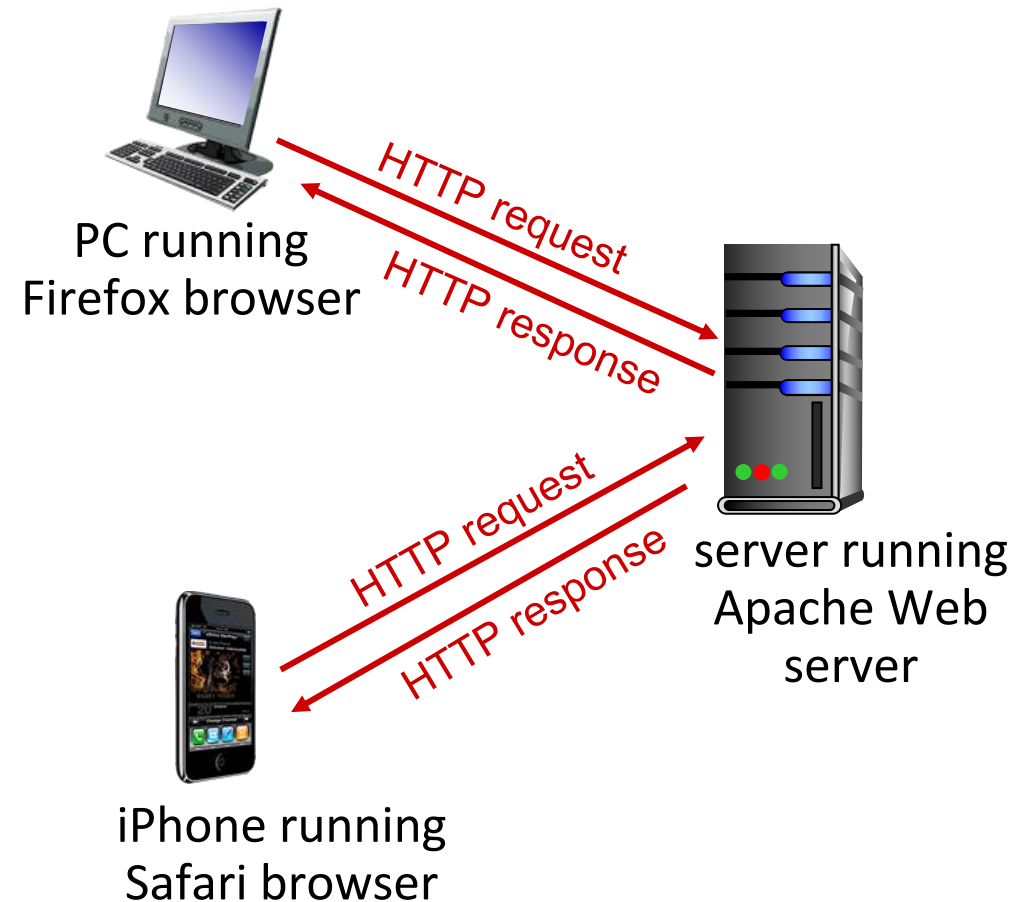
host name

path name

HTTP overview

HTTP: hypertext transfer protocol

- Web's application-layer protocol
- client/server model:
 - *client*: browser that requests, receives, (using HTTP protocol) and “displays” Web objects
 - *server*: Web server sends (using HTTP protocol) objects in response to requests



HTTP overview (continued)

HTTP uses TCP:

- client initiates TCP connection (creates socket) to server, port 80
- server accepts TCP connection from client
- HTTP messages (application-layer protocol messages) exchanged between browser (HTTP client) and Web server (HTTP server)
- TCP connection closed

HTTP is “stateless”

- server maintains *no* information about past client requests

aside
protocols that maintain “state” are complex!

- past history (state) must be maintained
- if server/client crashes, their views of “state” may be inconsistent, must be reconciled

HTTP connections: two types

Non-persistent HTTP

1. TCP connection opened
2. at most one object sent over TCP connection
3. TCP connection closed

downloading multiple objects required multiple connections

Persistent HTTP

- TCP connection opened to a server
- multiple objects can be sent over *single* TCP connection between client, and that server
- TCP connection closed

Application layer: overview

- Principles of network applications
- Web and HTTP
- E-mail, SMTP, IMAP
- The Domain Name System
DNS



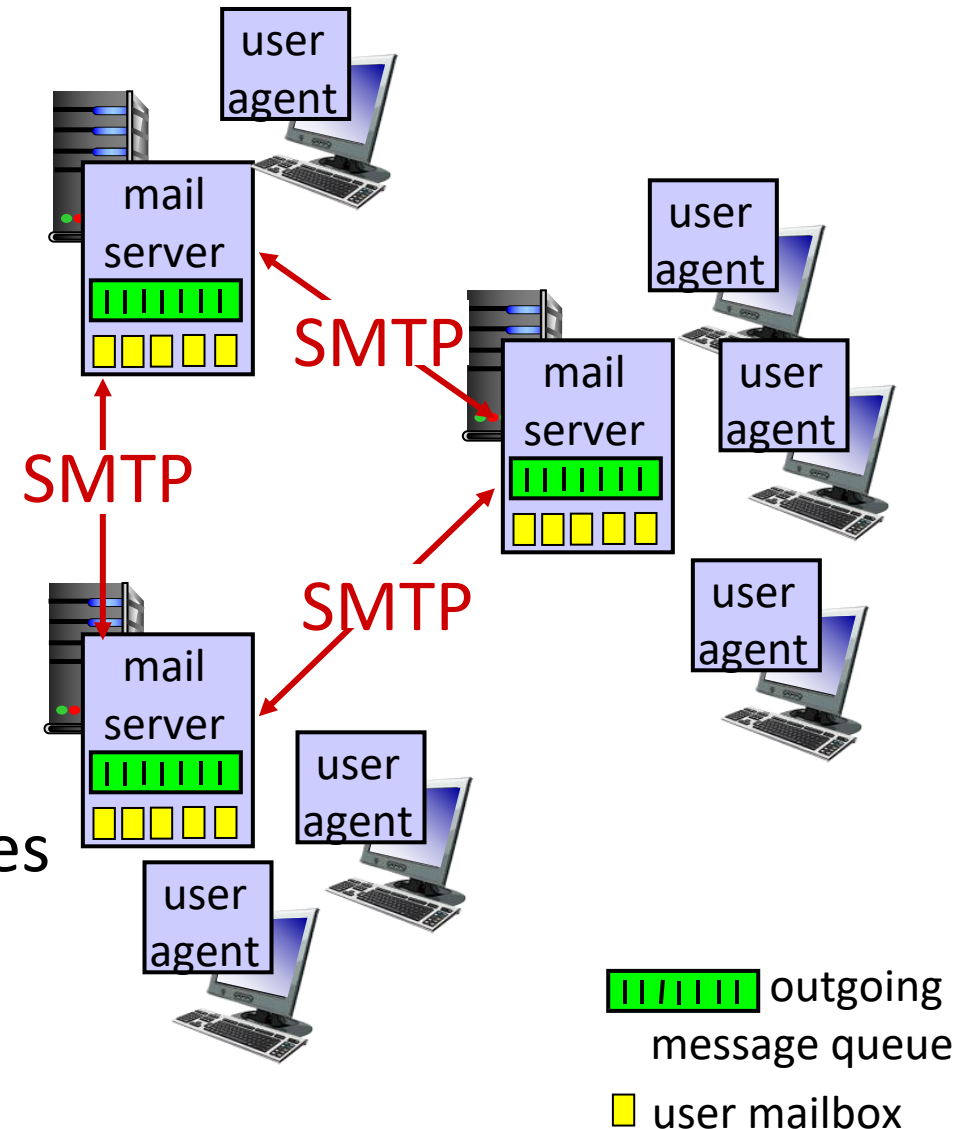
E-mail

Three major components:

- user agents
- mail servers
- simple mail transfer protocol: SMTP

User Agent

- a.k.a. “mail reader”
- composing, editing, reading mail messages
- e.g., Outlook, iPhone mail client
- outgoing, incoming messages stored on server



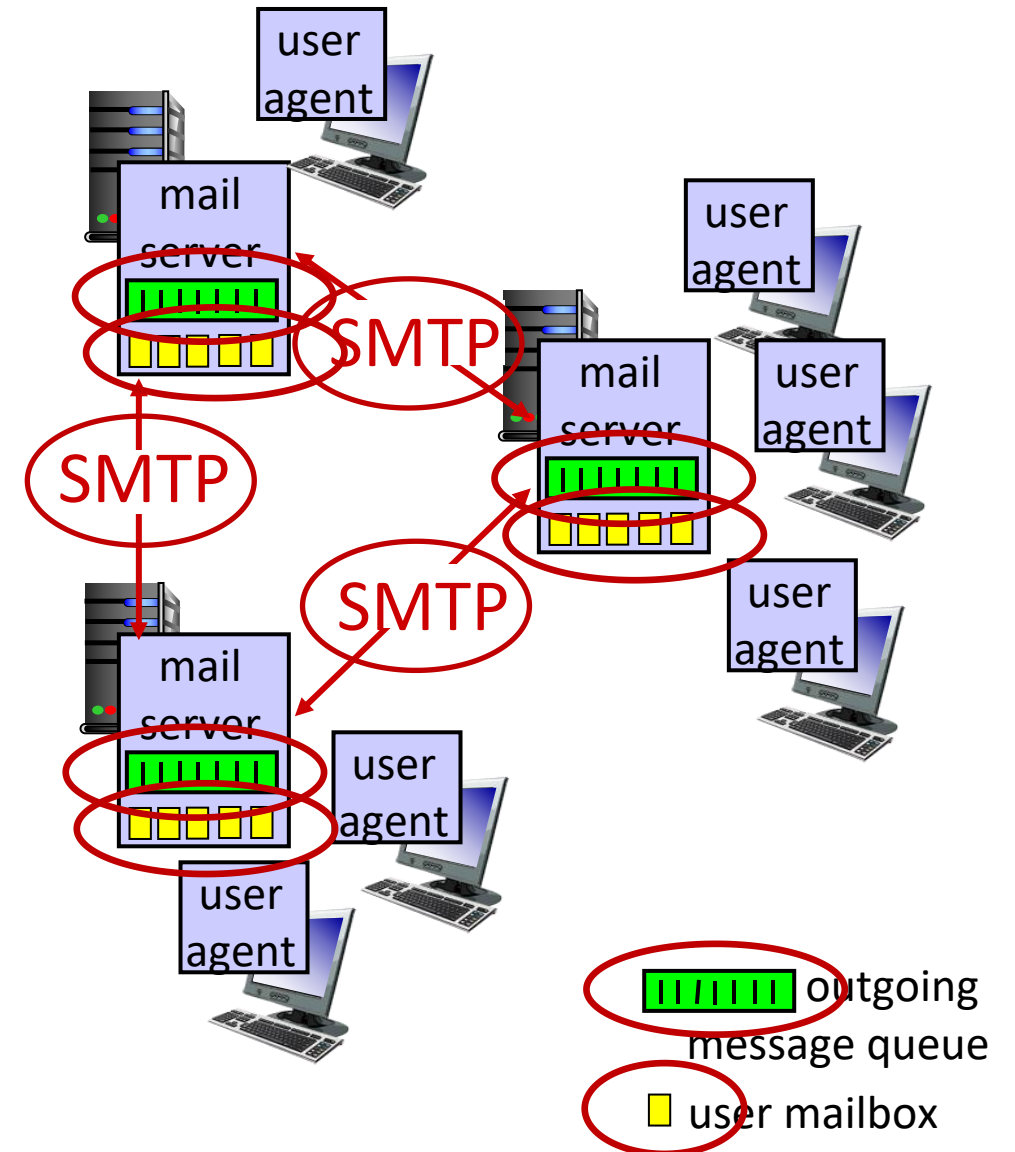
E-mail: mail servers

mail servers:

- *mailbox* contains incoming messages for user
- *message queue* of outgoing (to be sent) mail messages

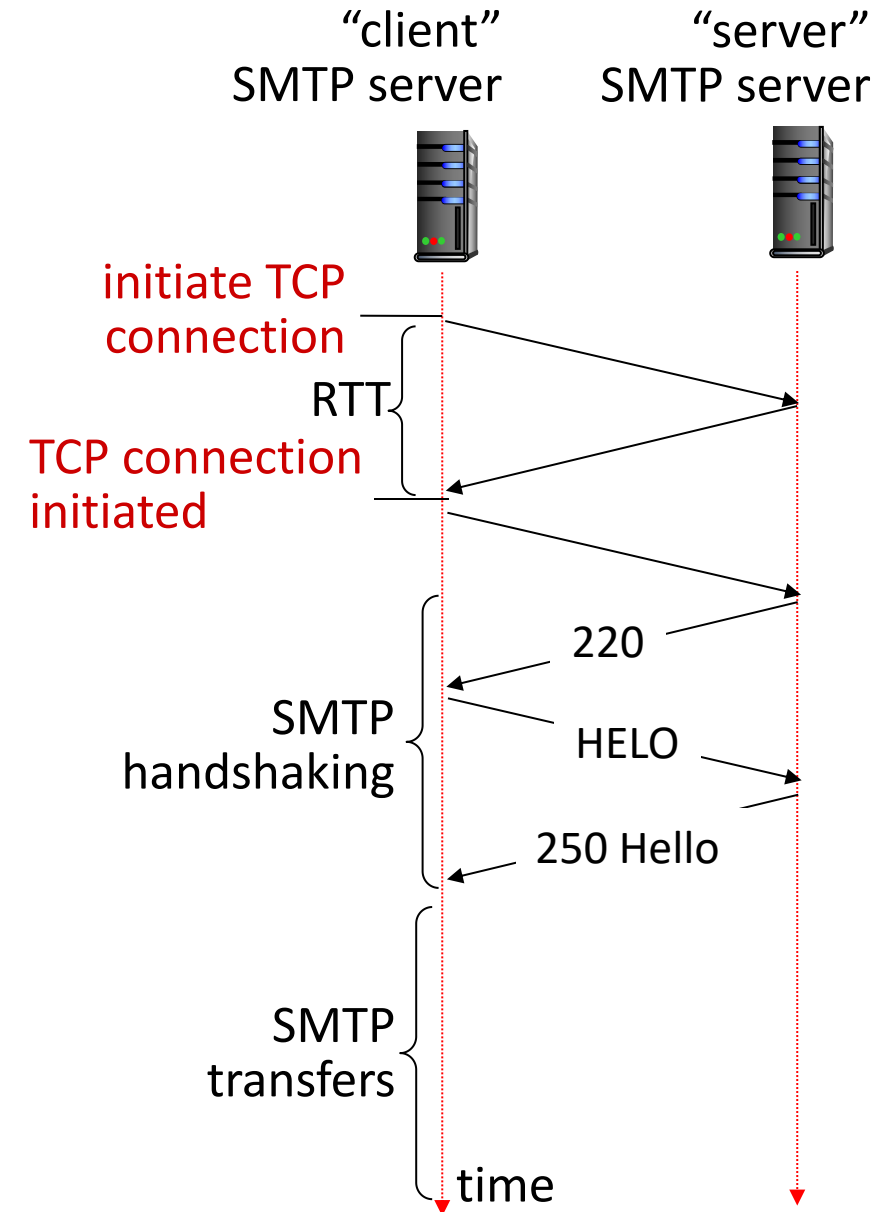
SMTP protocol between mail servers to send email messages

- **client**: sending mail server
- **“server”**: receiving mail server



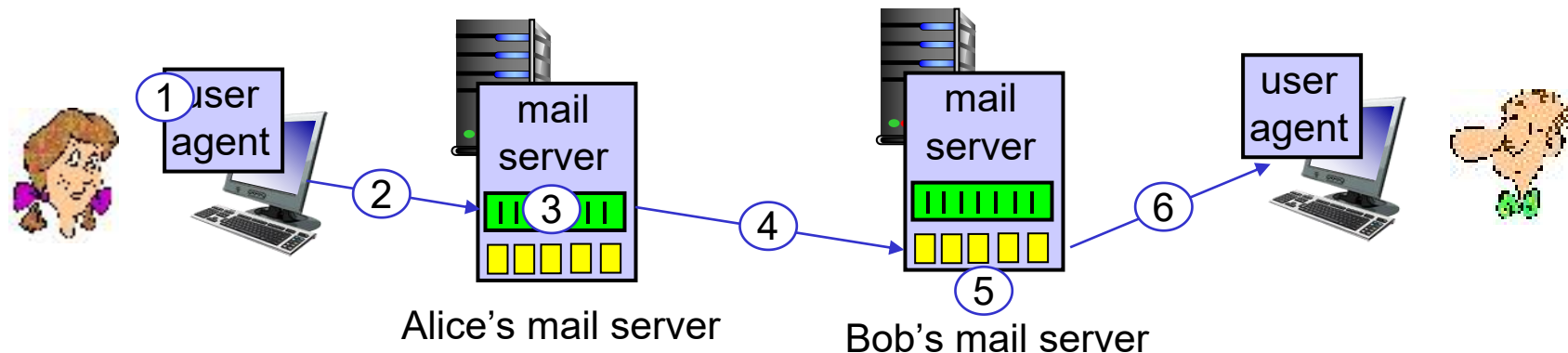
SMTP RFC (5321)

- uses TCP to reliably transfer email message from client (mail server initiating connection) to server, port 25
 - direct transfer: sending server (acting like client) to receiving server
- three phases of transfer
 - SMTP handshaking (greeting)
 - SMTP transfer of messages
 - SMTP closure
- command/response interaction (like HTTP)
 - **commands**: ASCII text
 - **response**: status code and phrase



Scenario: Alice sends e-mail to Bob

- 1) Alice uses UA to compose e-mail message "to" bob@some school.edu
- 2) Alice's UA sends message to her mail server using SMTP; message placed in message queue
- 3) client side of SMTP at mail server opens TCP connection with Bob's mail server
- 4) SMTP client sends Alice's message over the TCP connection
- 5) Bob's mail server places the message in Bob's mailbox
- 6) Bob invokes his user agent to read message

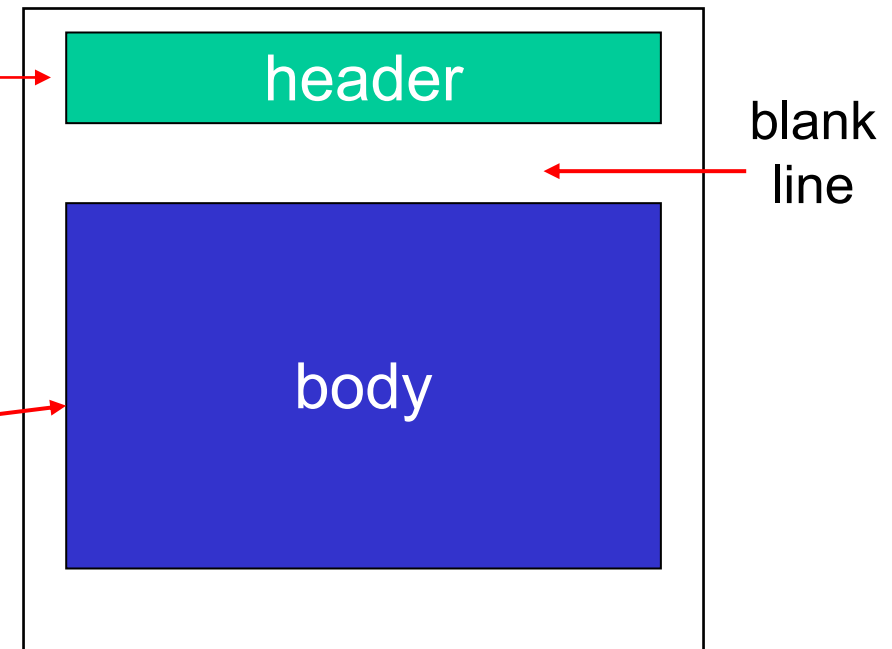


Mail message format

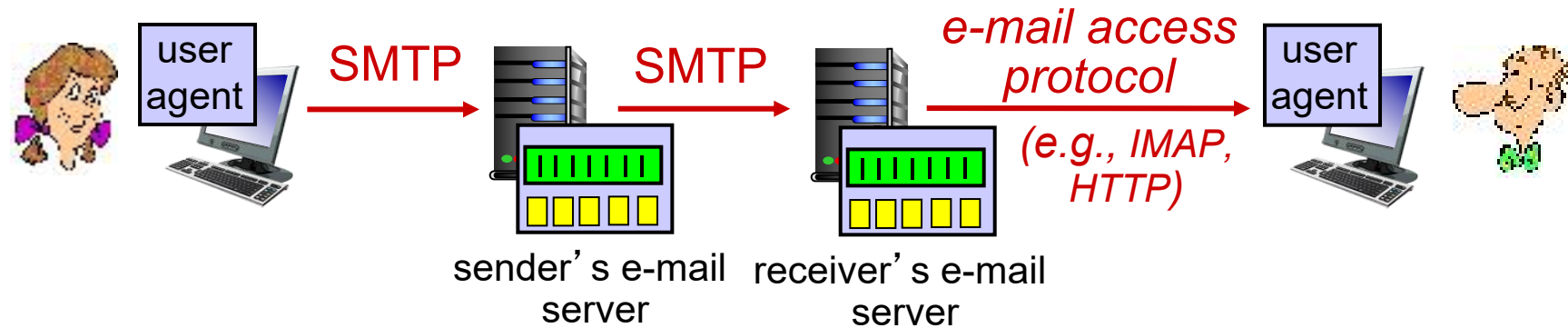
SMTP: protocol for exchanging e-mail messages, defined in RFC 5321 (like RFC 7231 defines HTTP)

RFC 2822 defines *syntax* for e-mail message itself (like HTML defines syntax for web documents)

- header lines, e.g.,
 - To:
 - From:
 - Subject:these lines, within the body of the email message area different from SMTP MAIL FROM:, RCPT TO: commands!
- Body: the “message” , ASCII characters only



Retrieving email: mail access protocols



- **SMTP:** delivery/storage of e-mail messages to receiver's server
- mail access protocol: retrieval from server
 - **IMAP:** Internet Mail Access Protocol [RFC 3501]: messages stored on server, IMAP provides retrieval, deletion, folders of stored messages on server
- **HTTP:** gmail, Hotmail, Yahoo!Mail, etc. provides web-based interface on top of SMTP (to send), IMAP (or POP) to retrieve e-mail messages

Application Layer: Overview

- Principles of network applications
- Web and HTTP
- E-mail, SMTP, IMAP
- The Domain Name System
DNS



DNS: Domain Name System

people: many identifiers:

- SSN, name, passport #

Internet hosts, routers:

- IP address (32 bit) - used for addressing datagrams
- “name”, e.g., cs.umass.edu - used by humans

Q: how to map between IP address and name, and vice versa ?

Domain Name System (DNS):

- *distributed database* implemented in hierarchy of many *name servers*
- *application-layer protocol*: hosts, DNS servers communicate to *resolve* names (address/name translation)
 - *note*: core Internet function, implemented as application-layer protocol
 - complexity at network’s “edge”

DNS: services, structure

DNS services:

- hostname-to-IP-address translation
- host aliasing
 - canonical, alias names
- mail server aliasing
- load distribution
 - replicated Web servers: many IP addresses correspond to one name

Q: Why not centralize DNS?

- single point of failure
- traffic volume
- distant centralized database
- maintenance

A: doesn't scale!

- Comcast DNS servers alone: 600B DNS queries/day
- Akamai DNS servers alone: 2.2T DNS queries/day

Thinking about the DNS

humongous distributed database:

- ~ billion records, each simple

handles many *trillions* of queries/day:

- *many* more reads than writes
- *performance matters*: almost every Internet transaction interacts with DNS - msec count!

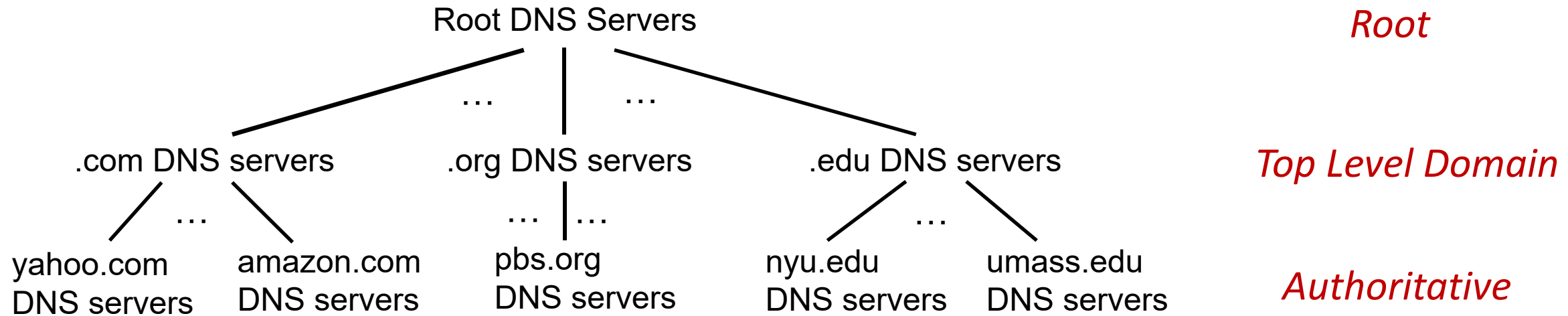
organizationally, physically decentralized:

- millions of different organizations responsible for their records

“bulletproof”: reliability, security



DNS: a distributed, hierarchical database

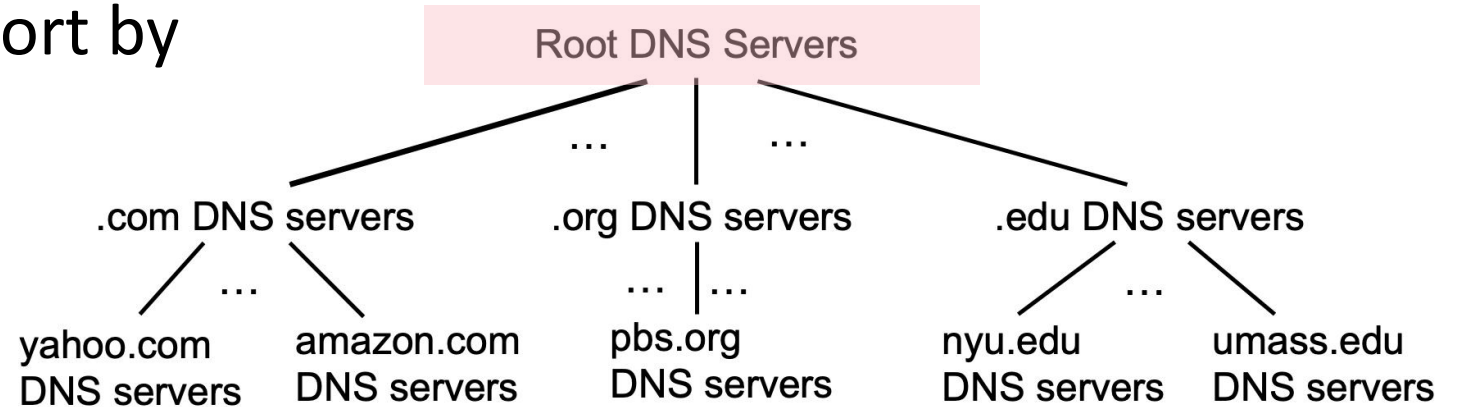


Client wants IP address for `www.amazon.com`; 1st approximation:

- client queries root server to find `.com` DNS server
- client queries `.com` DNS server to get `amazon.com` DNS server
- client queries `amazon.com` DNS server to get IP address for `www.amazon.com`

DNS: root name servers

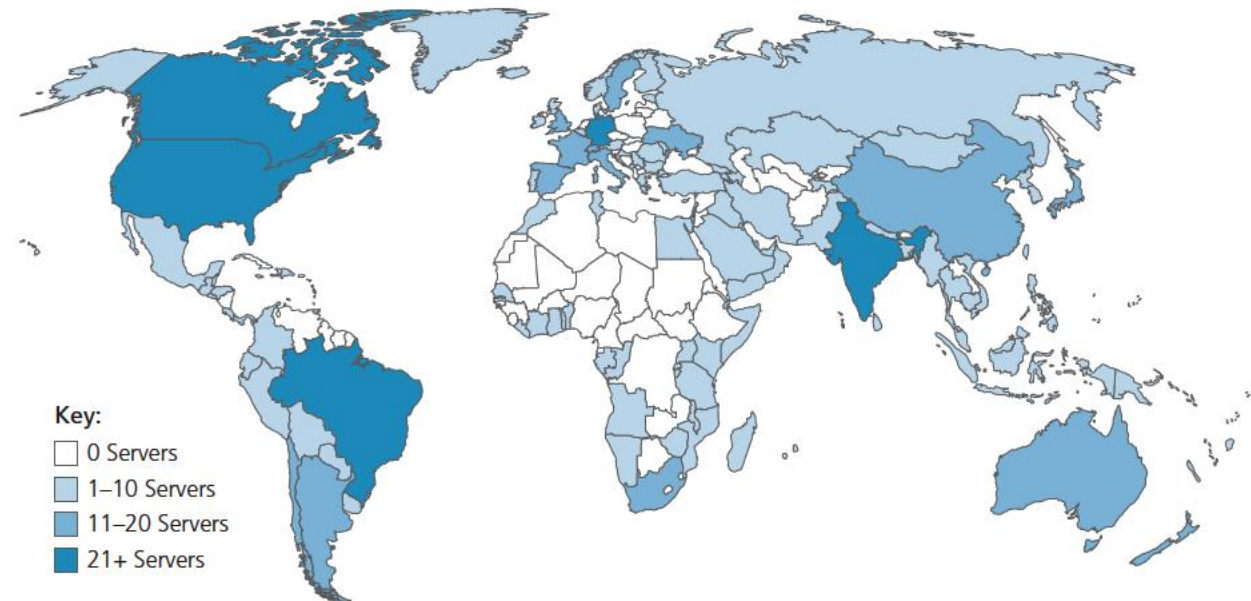
- official, contact-of-last-resort by name servers that can not resolve name



DNS: root name servers

- official, contact-of-last-resort by name servers that can not resolve name
- *incredibly important* Internet function
 - Internet couldn't function without it!
 - DNSSEC – provides security (authentication, message integrity)
- ICANN (Internet Corporation for Assigned Names and Numbers) manages root DNS domain

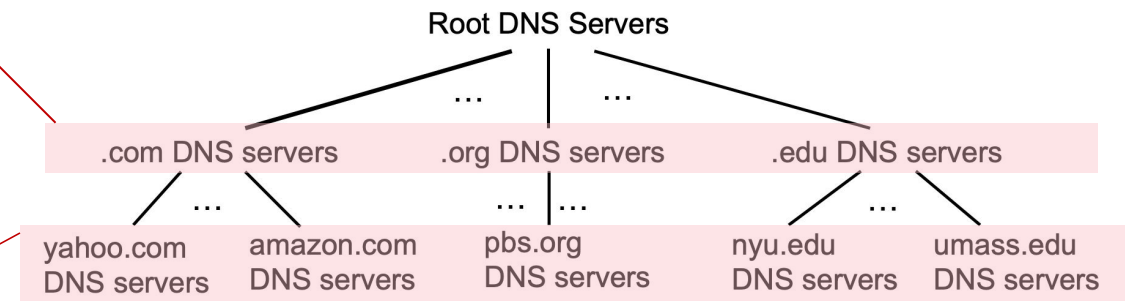
13 logical root name “servers”
worldwide each “server” replicated
many times (~200 servers in US)



Top-Level Domain, and authoritative servers

Top-Level Domain (TLD) servers:

- responsible for .com, .org, .net, .edu, .aero, .jobs, .museums, and all top-level country domains, e.g.: .cn, .uk, .fr, .ca, .jp
- Network Solutions: authoritative registry for .com, .net TLD
- Educause: .edu TLD



authoritative DNS servers:

- organization's own DNS server(s), providing authoritative hostname to IP mappings for organization's named hosts
- can be maintained by organization or service provider

Local DNS name servers

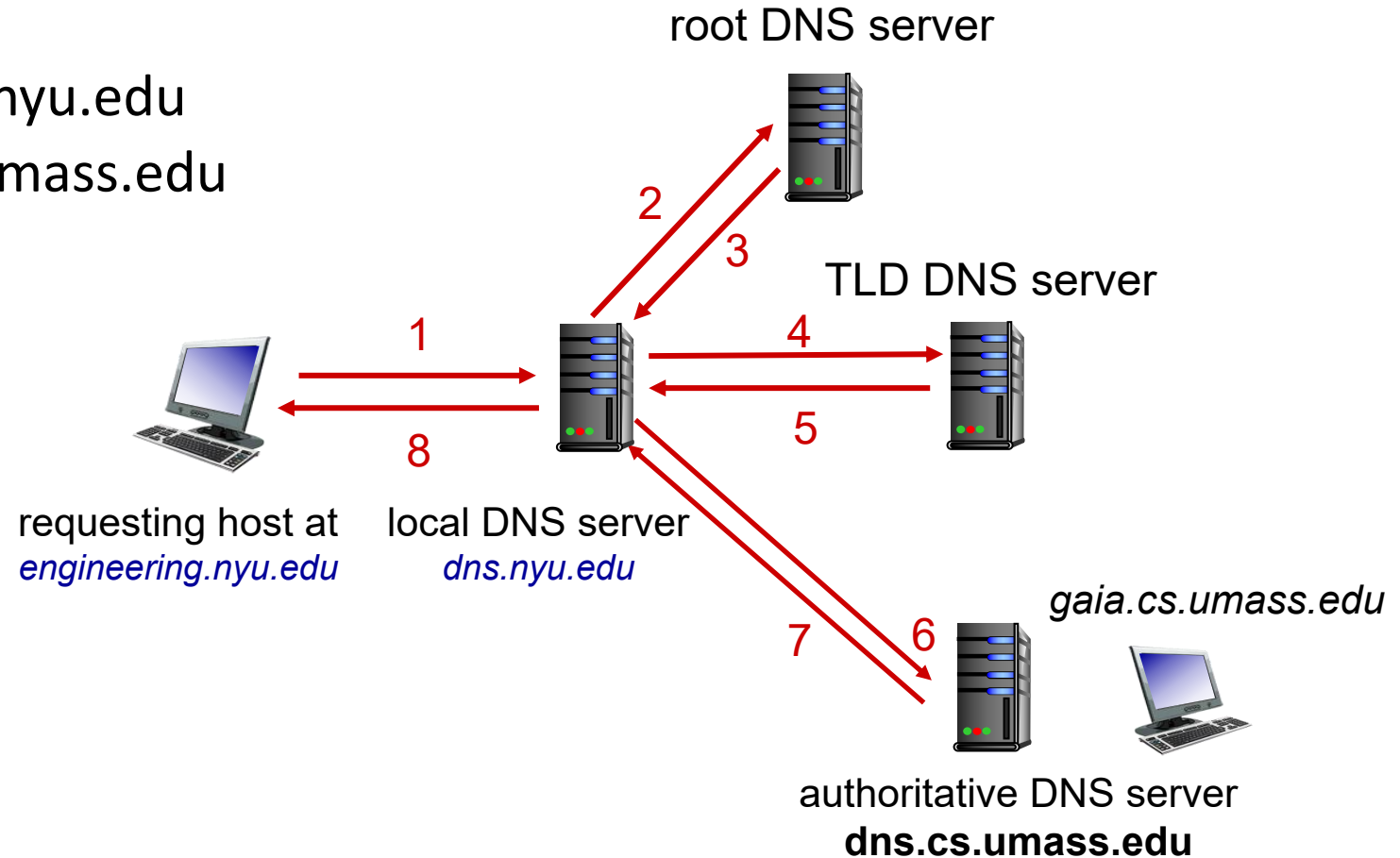
- when host makes DNS query, it is sent to its *local* DNS server
 - Local DNS server returns reply, answering:
 - from its local cache of recent name-to-address translation pairs (possibly out of date!)
 - forwarding request into DNS hierarchy for resolution
 - each ISP has local DNS name server; to find yours:
 - MacOS: `% scutil --dns`
 - Windows: `>ipconfig /all`
- local DNS server doesn't strictly belong to hierarchy

DNS name resolution: iterated query

Example: host at `engineering.nyu.edu` wants IP address for `gaia.cs.umass.edu`

Iterated query:

- contacted server replies with name of server to contact
- “I don’t know this name, but ask this server”

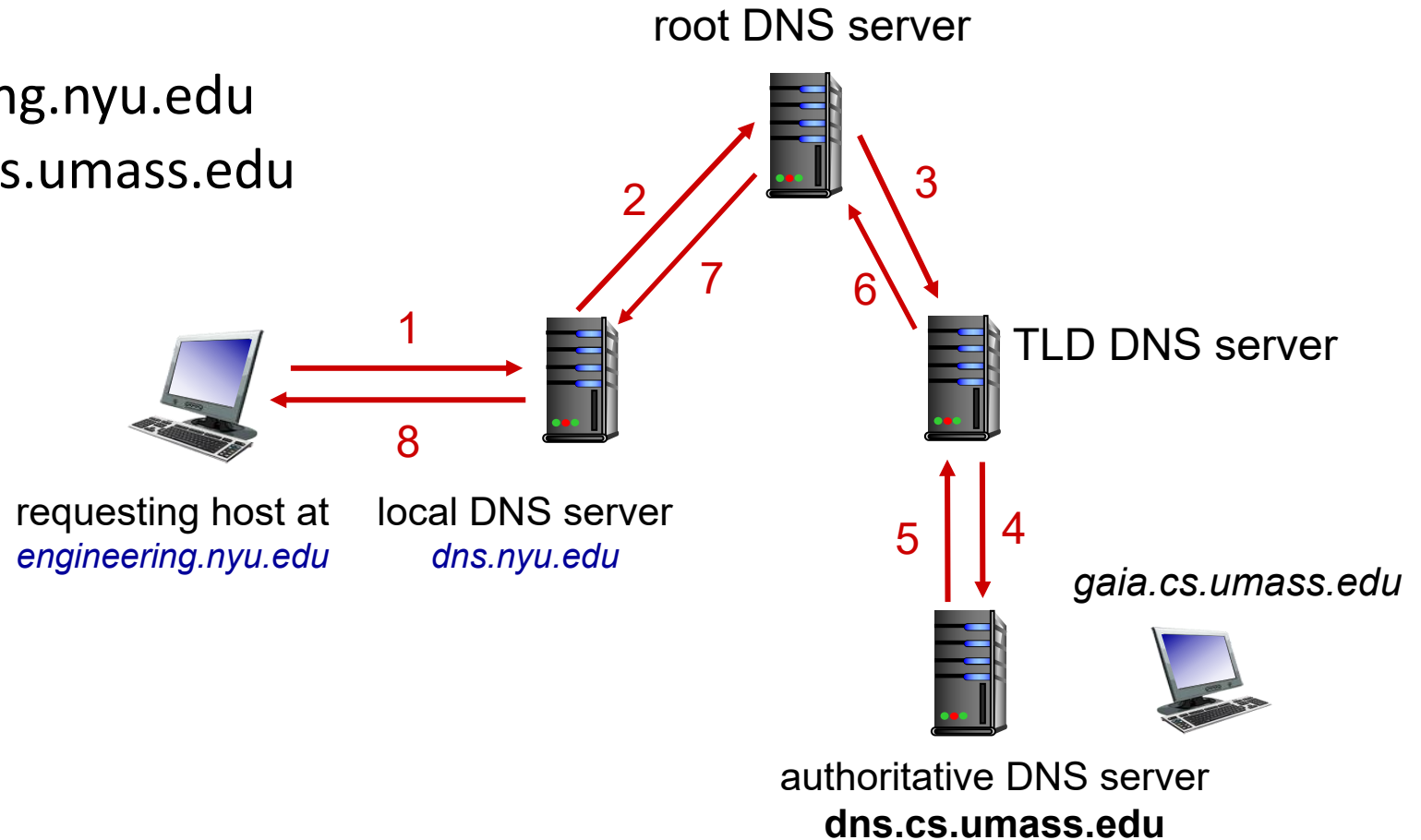


DNS name resolution: recursive query

Example: host at `engineering.nyu.edu` wants IP address for `gaia.cs.umass.edu`

Recursive query:

- puts burden of name resolution on contacted name server
- heavy load at upper levels of hierarchy?



Caching DNS Information

- once (any) name server learns mapping, it *caches* mapping, and *immediately* returns a cached mapping in response to a query
 - caching improves response time
 - cache entries timeout (disappear) after some time (TTL)
 - TLD servers typically cached in local name servers
- cached entries may be *out-of-date*
 - if named host changes IP address, may not be known Internet-wide until all TTLs expire!
 - *best-effort name-to-address translation!*

DNS records

DNS: distributed database storing resource records (RR)

RR format: (name, value, type, ttl)

type=A

- name is hostname
- value is IP address

type=NS

- name is domain (e.g., foo.com)
- value is hostname of authoritative name server for this domain

type=CNAME

- name is alias name for some “canonical” (the real) name
- www.ibm.com is really servereast.backup2.ibm.com
- value is canonical name

type=MX

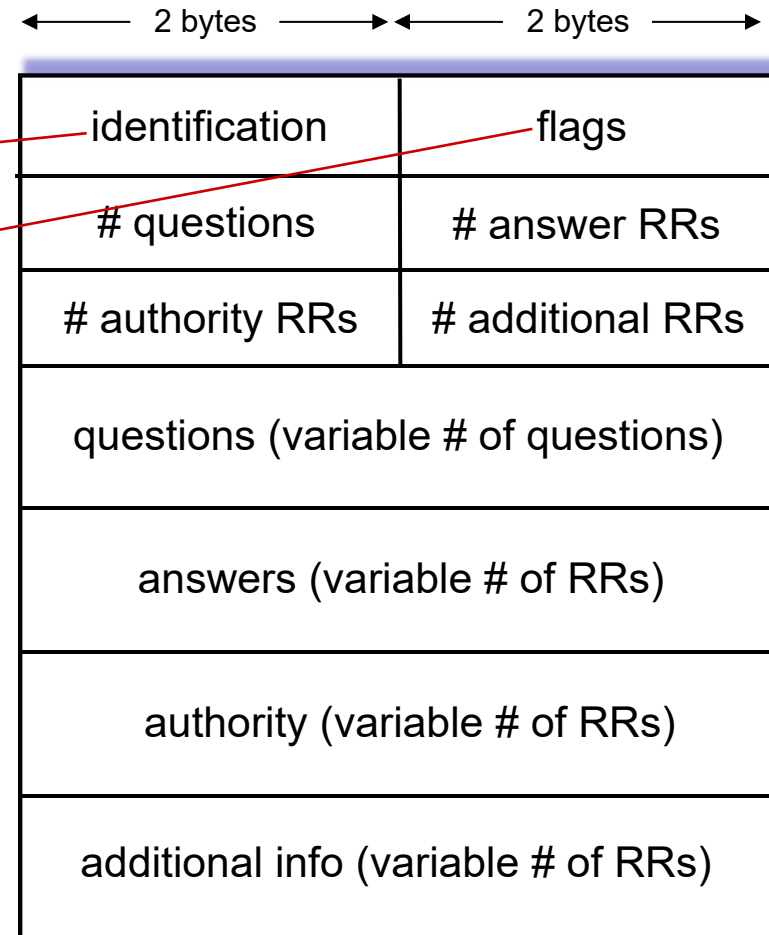
- value is name of SMTP mail server associated with name

DNS protocol messages

DNS *query* and *reply* messages, both have same *format*:

message header:

- **identification**: 16 bit # for query, reply to query uses same #
- **flags**:
 - query or reply
 - recursion desired
 - recursion available
 - reply is authoritative



DNS protocol messages

DNS *query* and *reply* messages, both have same *format*:

← 2 bytes → ← 2 bytes →

identification	flags
# questions	# answer RRs
# authority RRs	# additional RRs
questions (variable # of questions)	
answers (variable # of RRs)	
authority (variable # of RRs)	
additional info (variable # of RRs)	

name, type fields for a query

RRs in response to query

records for authoritative servers

additional “helpful” info that may
be used

Getting your info into the DNS

example: new startup “Network Utopia”

- register name networkutopia.com at *DNS registrar* (e.g., Network Solutions)
 - provide names, IP addresses of authoritative name server (primary and secondary)
 - registrar inserts NS, A RRs into .com TLD server:
`(networkutopia.com, dns1.networkutopia.com, NS)`
`(dns1.networkutopia.com, 212.212.212.1, A)`
- create authoritative server locally with IP address `212.212.212.1`
 - type A record for `www.networkutopia.com`
 - type MX record for `networkutopia.com`

DNS security

DDoS attacks

- bombard root servers with traffic
 - not successful to date
 - traffic filtering
 - local DNS servers cache IPs of TLD servers, allowing root server bypass
- bombard TLD servers
 - potentially more dangerous

Spoofing attacks

- intercept DNS queries, returning bogus replies
 - DNS cache poisoning
 - RFC 4033: DNSSEC authentication services